AI Multi-Agent System Documentation

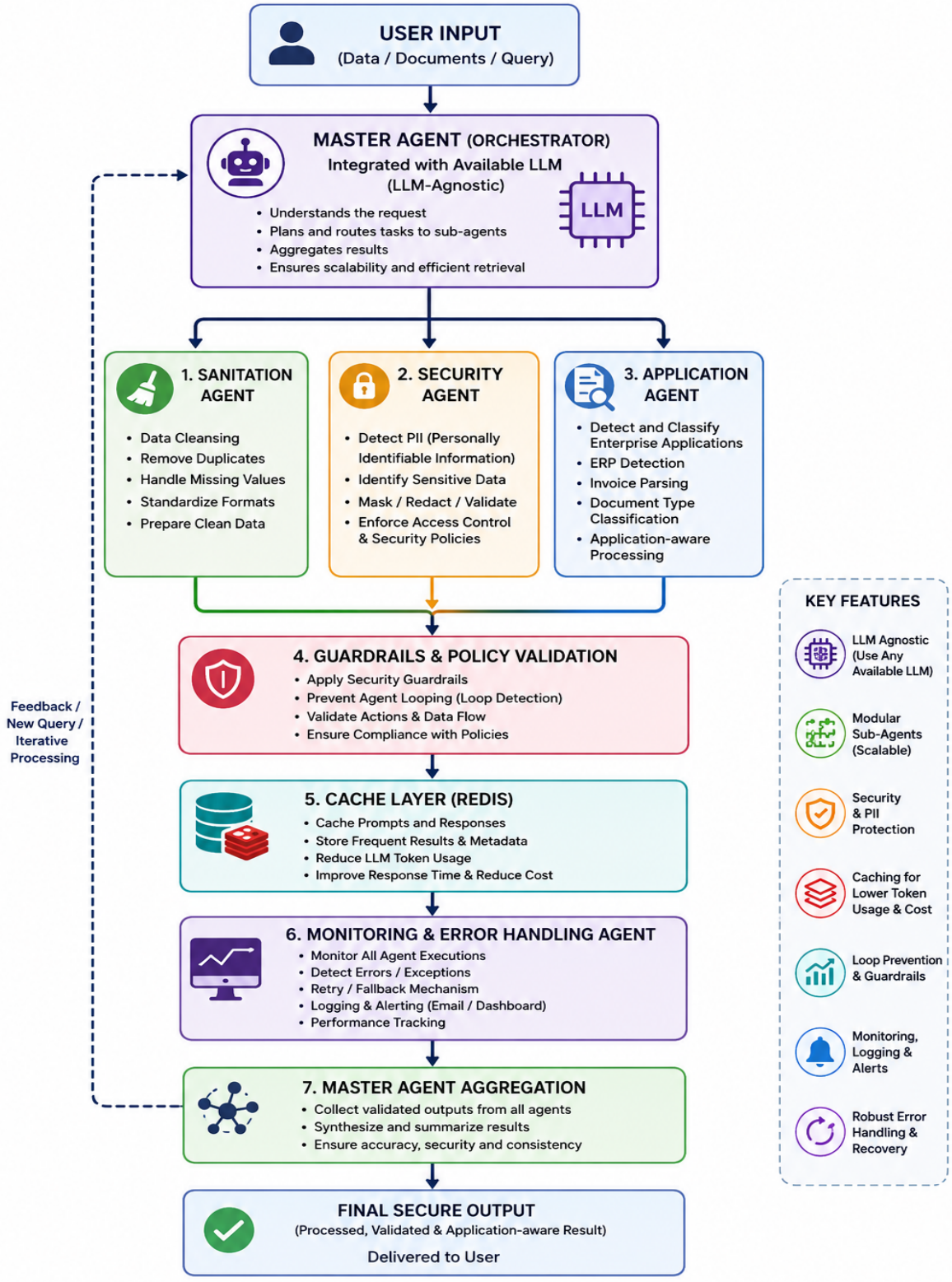
Enterprise AI Document Processing Platform

Project Overview

This project presents a scalable AI Multi-Agent architecture for processing enterprise documents securely and efficiently. A Master Agent orchestrates multiple specialized agents responsible for data cleansing, security validation, and application-aware document analysis. The architecture is LLM-agnostic, enabling integration with any available Large Language Model while maintaining modularity and scalability. Redis-based caching reduces latency and token usage, while guardrails and monitoring ensure secure and reliable execution.

Objectives

• Build a modular multi-agent architecture.
• Support any available LLM without vendor lock-in.
• Clean and standardize enterprise data.
• Detect PII and sensitive information.
• Classify ERP, Invoice and other enterprise documents.
• Reduce response time and LLM cost using Redis caching.
• Ensure security, observability and fault tolerance.


USER INPUT
(Data / Documents / Query)
MASTER AGENT (ORCHESTRATOR)
Integrated with Available LLM
(LLM-Agnostic)
LLM
• Understands the request
• Plans and routes tasks to sub-agents
• Aggregates results
• Ensures scalability and efficient retrieval
1. SANITATION AGENT
• Data Cleansing
• Remove Duplicates
• Handle Missing Values
• Standardize Formats
• Prepare Clean Data
2. SECURITY AGENT
• Detect PII (Personally Identifiable Information)
• Identify Sensitive Data
• Mask / Redact / Validate
• Enforce Access Control & Security Policies
3. APPLICATION AGENT
• Detect and Classify Enterprise Applications
• ERP Detection
• Invoice Parsing
• Document Type Classification
• Application-aware Processing
4. GUARDRAILS & POLICY VALIDATION
• Apply Security Guardrails
• Prevent Agent Looping (Loop Detection)
• Validate Actions & Data Flow
• Ensure Compliance with Policies
5. CACHE LAYER (REDIS)
• Cache Prompts and Responses
• Store Frequent Results & Metadata
• Reduce LLM Token Usage
• Improve Response Time & Reduce Cost
6. MONITORING & ERROR HANDLING AGENT
• Monitor All Agent Executions
• Detect Errors / Exceptions
• Retry / Fallback Mechanism
• Logging & Alerting (Email / Dashboard)
• Performance Tracking
7. MASTER AGENT AGGREGATION
• Collect validated outputs from all agents
• Synthesize and summarize results
• Ensure accuracy, security and consistency
FINAL SECURE OUTPUT
(Processed, Validated & Application-aware Result)
Delivered to User
Feedback / New Query / Iterative Processing
KEY FEATURES
LLM Agnostic (Use Any Available LLM)
Modular Sub-Agents (Scalable)
Security & PII Protection
Caching for Lower Token Usage & Cost
Loop Prevention & Guardrails
Monitoring, Logging & Alerts
Robust Error Handling & Recovery

Architecture & Workflow

1. User Request: User submits data or documents.

2. Master Agent: Receives the request, plans execution, routes tasks to sub-agents, aggregates outputs and generates the final response.

3. Sanitation Agent: Cleanses data, removes duplicates, handles missing values and standardizes formats.

4. Security Agent: Detects PII and sensitive data, masks confidential information and enforces access policies.

5. Application Agent: Identifies ERP, invoices and enterprise documents for application-specific processing.

6. Guardrails: Validate execution, enforce security rules and prevent recursive agent looping.

7. Redis Cache: Stores prompts and responses to improve retrieval speed, reduce token usage and lower inference cost.

8. Monitoring & Error Handling Agent: Tracks failures, retries failed tasks, logs events and sends alerts for operational visibility.

9. Final Output: The Master Agent combines validated results into a secure and accurate response.

Design Considerations & Key Features

LLM Agnostic: The solution supports any available LLM and avoids dependence on a specific model.

Security Guardrails: Guardrails ensure policy compliance, prevent unauthorized access and stop agent looping during orchestration.

Caching Strategy: Redis caching minimizes repeated LLM calls, improves latency, manages token consumption and reduces operational cost.

Error Handling: A dedicated Monitoring & Error Handling Agent detects failures, records logs, triggers alerts and performs retry/fallback mechanisms for higher reliability.

Scalability: New specialized agents can be added without changing the Master Agent architecture, making the platform suitable for enterprise growth.

Technology Stack: Python, LangGraph, FastAPI, Redis, Available LLM, Docker (optional), Logging & Monitoring Frameworks.

Future Scope: Add RAG with vector databases, Human-in-the-Loop approvals, analytics dashboards, workflow automation and additional domain-specific agents.